

SH Day 3: Regularization & Variable Selection — Voter Turnout & Term Limits (Political Science)

Intro to Machine Learning · Bruce A. Desmarais

July 23, 2026

Published study. Kuhlmann, R. & Lewis, D. C. (2017). “Legislative Term Limits and Voter Turnout.” *State Politics & Policy Quarterly* 17(4):372–392. Data: UNC Dataverse doi: 10.15139/S3/POGNLC. The original analysis is an **OLS** regression of state lower-chamber turnout on institutional predictors (Table 2, Col. 1). We reproduce that model, then extend it with **regularization and variable selection** — ridge, lasso, elastic net (**glmnet**) and best-subset selection (**abess**) — on the same outcome and the same 13 predictors.

1 Background

Kuhlmann and Lewis ask whether **legislative term limits shape voter turnout** in state legislative elections. Term limits reshape the electoral environment – forcing open-seat contests and changing candidate experience and the information available to voters – with theoretically ambiguous effects on participation. They use **OLS** to estimate the association between the strength of a state’s term limits and its lower-chamber turnout, holding constant the major institutional and demographic determinants of turnout (registration rules, professionalism, party competition, election timing, income, diversity). In the reproduced Table 2 model the **term-limitedness coefficient is positive and statistically significant** – more term-limited legislatures see modestly higher turnout – though the largest effects are presidential-election timing and ease of registration.

2 Data and codebook

Unit of analysis: a U.S. state lower-chamber legislative election (a **state-year**); $n = 486$.

Variable	Definition
turnout	lower-chamber election turnout rate, % of the voting-age population (outcome)
tlness	term-limitedness: strength/bindingness of legislative term limits
lcr2	log legislator-to-citizen ratio (per 100,000 residents)
mmds	share of legislative seats elected from multimember districts
regdifficulty	voter-registration closing date (days before the election)
straight	1 if the state offers straight-party-ticket voting
iuse2	log number of ballot initiatives on the ballot
squire	Squire index of legislative professionalism
ranney	Ranney index of interparty electoral competition
income2	per-capita income (thousands of dollars)
diversity3	Herfindahl index of racial/ethnic diversity

Variable	Definition
preselec	1 in presidential-election years
redistrict	1 if districts were newly redrawn (post-redistricting)
year	election year

3 Descriptive analysis

Before modeling, we profile the data thoroughly: the sample, the outcome, the predictors one at a time, missingness, each predictor's bivariate link to turnout, and the collinearity structure. This EDA motivates every choice in the reproduction that follows.

```
d <- read.delim(paste0(
  "https://raw.githubusercontent.com/bdesmarais/intro",
  "-ml-statistical-horizons-4day/main/tutorials/day3_",
  "regularization_and_selection/01_turnout/data/",
  "turnout_state.tab"))
mvars <- c("turnout", "tlness", "lcr2", "mmds", "regdifficulty", "straight", "iuse2",
  "squire", "ranney", "income2", "diversity3", "preselec", "redistrict", "year")
d <- as.data.frame(na.omit(d[, mvars]))
preds <- setdiff(mvars, "turnout")

library(ggplot2)
library(dplyr)
library(tidyr)
library(e1071)
theme_set(theme_minimal(base_size = 9))

binvars <- c("straight", "preselec", "redistrict") # 0/1 indicators
numpred <- setdiff(preds, binvars) # continuous predictors
```

3.1 Dataset and provenance

The **unit of analysis** is a U.S. state lower-chamber legislative election (a state-year). After restricting to the modeling variables and dropping incomplete rows we have **n = 486** state-years and **14** variables: one continuous outcome (**turnout**), 10 continuous predictors, and 3 binary indicators. The data are Kuhlmann & Lewis's replication file (UNC Dataverse doi:10.15139/S3/POGNLC), assembling state turnout with institutional and demographic covariates across recent election cycles.

```
dict <- data.frame(
  variable = mvars,
  role = c("outcome", rep("predictor", length(preds))),
  type = c("continuous", "continuous", "continuous", "continuous", "continuous",
    "binary", "continuous", "continuous", "continuous", "continuous",
    "continuous", "binary", "binary", "continuous"),
  description = c(
    "lower-chamber turnout, % of voting-age population",
    "term-limitedness (strength/bindingness of limits)",
    "log legislator-to-citizen ratio (per 100k)",
    "share of seats from multimember districts",
    "registration closing date (days before election)",
    "1 = straight-party-ticket voting offered",
    "log number of ballot initiatives",
    "Squire index of legislative professionalism",
```

```

"Ranney index of interparty competition",
"per-capita income ($1000s)",
"Herfindahl index of racial/ethnic diversity",
"1 = presidential-election year",
"1 = newly redrawn (post-redistricting)",
"election year"))
knitr::kable(dict, caption = "Variable dictionary: role, type, and coding.")

```

Table 1: Variable dictionary: role, type, and coding.

variable	role	type	description
turnout	outcome	continuous	lower-chamber turnout, % of voting-age population
tlness	predictor	continuous	term-limitedness (strength/bindingness of limits)
lcr2	predictor	continuous	log legislator-to-citizen ratio (per 100k)
mmds	predictor	continuous	share of seats from multimember districts
regdifficulty	predictor	continuous	registration closing date (days before election)
straight	predictor	binary	1 = straight-party-ticket voting offered
iuse2	predictor	continuous	log number of ballot initiatives
squire	predictor	continuous	Squire index of legislative professionalism
ranney	predictor	continuous	Ranney index of interparty competition
income2	predictor	continuous	per-capita income (\$1000s)
diversity3	predictor	continuous	Herfindahl index of racial/ethnic diversity
preselec	predictor	binary	1 = presidential-election year
redistrict	predictor	binary	1 = newly redrawn (post-redistricting)
year	predictor	continuous	election year

3.2 The outcome in depth

```

y <- d$turnout
ostat <- data.frame(
  n = length(y), mean = mean(y), sd = sd(y), min = min(y),
  Q1 = quantile(y, .25), median = median(y), Q3 = quantile(y, .75),
  max = max(y), IQR = IQR(y), skew = skewness(y), kurt = kurtosis(y))
knitr::kable(round(ostat, 2), row.names = FALSE,
  caption = "Turnout (%): summary statistics.")

```

Table 2: Turnout (%): summary statistics.

n	mean	sd	min	Q1	median	Q3	max	IQR	skew	kurt
486	43.67	10.52	14.67	36.13	42.72	51.85	74.6	15.73	0.05	-0.42

```

ggplot(d, aes(turnout)) +
  geom_histogram(aes(y = after_stat(density)), bins = 30,
    fill = "#a6cee3", colour = "white") +
  geom_density(colour = "#1f78b4", linewidth = 0.8) +
  geom_vline(xintercept = mean(y), linetype = 2, colour = "#e31a1c") +
  labs(x = "turnout (% of voting-age population)", y = "density")

```

Turnout is close to **symmetric** (skewness 0.05) and mildly **platykurtic** (kurtosis -0.42, i.e. thinner tails than a normal), ranging from about 15% to 75% with no floor/ceiling pile-up and no extreme outliers. Because

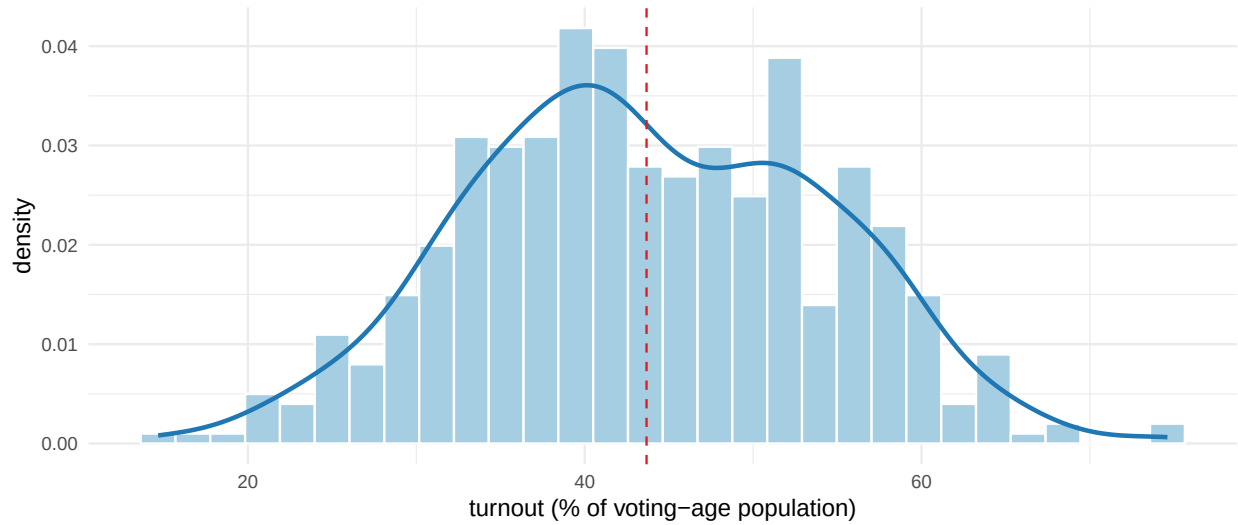


Figure 1: Distribution of state lower-chamber turnout, with density overlay and mean (dashed).

the distribution is already well-behaved, **no transformation of the outcome is warranted** — OLS on turnout in its natural units is appropriate.

3.3 Univariate predictor distributions

```
ptab <- t(sapply(d[numpred], function(x) c(
  n = sum(!is.na(x)), n_miss = sum(is.na(x)), mean = mean(x), sd = sd(x),
  min = min(x), Q1 = quantile(x, .25), median = median(x),
  Q3 = quantile(x, .75), max = max(x), skew = skewness(x))))
colnames(ptab) <- c("n", "n_miss", "mean", "sd", "min", "Q1",
  "median", "Q3", "max", "skew")
knitr::kable(round(ptab, 2),
  caption = "Continuous predictors: summary statistics.")
```

Table 3: Continuous predictors: summary statistics.

	n	n_miss	mean	sd	min	Q1	median	Q3	max	skew
tlness	486	0	0.08	0.28	0.00	0.00	0.00	0.00	1.57	4.13
lcr2	486	0	1.35	0.90	-1.05	0.77	1.28	1.94	3.61	0.16
mmms	486	0	0.16	0.34	0.00	0.00	0.00	0.00	1.00	1.91
regdifficulty	486	0	20.81	10.02	0.00	15.00	23.00	30.00	30.00	-0.96
iuse2	486	0	0.77	1.13	0.00	0.00	0.00	1.69	4.00	1.09
squire	486	0	0.20	0.13	0.05	0.12	0.16	0.23	0.66	1.67
ranney	486	0	0.89	0.09	0.63	0.83	0.91	0.96	1.00	-0.65
income2	486	0	27.89	8.74	12.53	20.82	26.85	33.98	56.96	0.51
diversity3	486	0	33.91	15.55	4.40	22.29	31.77	47.15	73.20	0.22
year	486	0	1998.99	6.87	1988.00	1994.00	1998.50	2004.75	2010.00	0.01

```
d %>% select(all_of(numpred)) %>%
  pivot_longer(everything(), names_to = "var", values_to = "val") %>%
  ggplot(aes(val)) +
  geom_histogram(bins = 22, fill = "#b2df8a", colour = "white") +
```

```
facet_wrap(~ var, scales = "free", ncol = 4) +
labs(x = NULL, y = "count")
```

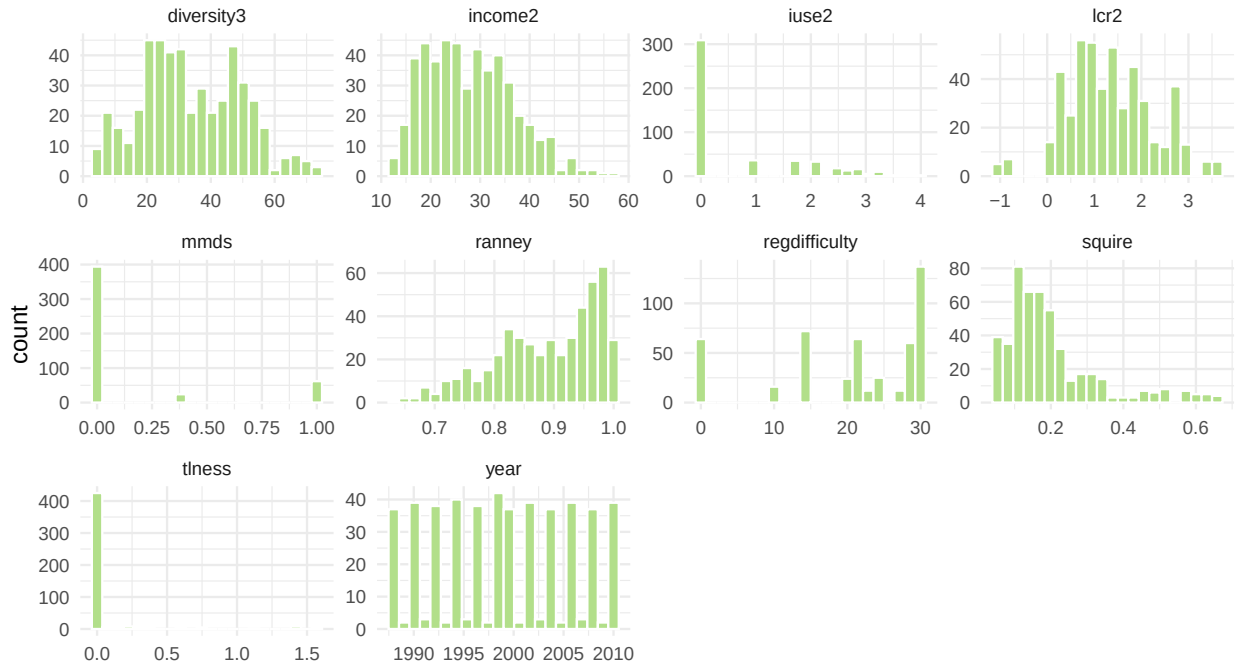


Figure 2: Distributions of the continuous predictors (faceted, free x-axes).

Most predictors are moderately behaved; **squire** (professionalism) and **mmds** (multimember-district share) are right-skewed, reflecting a few highly professional legislatures and states leaning on multimember districts. The three binary indicators split as follows:

```
btab <- do.call(rbind, lapply(binvars, function(v) {
  t <- table(d[[v]])
  data.frame(variable = v, n_0 = t["0"], n_1 = t["1"],
    pct_1 = round(100 * mean(d[[v]]), 1))
}))
knitr::kable(btab, row.names = FALSE,
  caption = "Binary predictors: frequencies and share coded 1.")
```

Table 4: Binary predictors: frequencies and share coded 1.

variable	n_0	n_1	pct_1
straight	303	183	37.7
preselec	262	224	46.1
redistrict	379	107	22.0

```
d %>% select(all_of(binvars)) %>%
  pivot_longer(everything(), names_to = "var", values_to = "val") %>%
  count(var, val) %>%
  ggplot(aes(factor(val), n)) +
  geom_col(fill = "#fb9a99") +
  facet_wrap(~ var) +
  labs(x = "value (0/1)", y = "count")
```

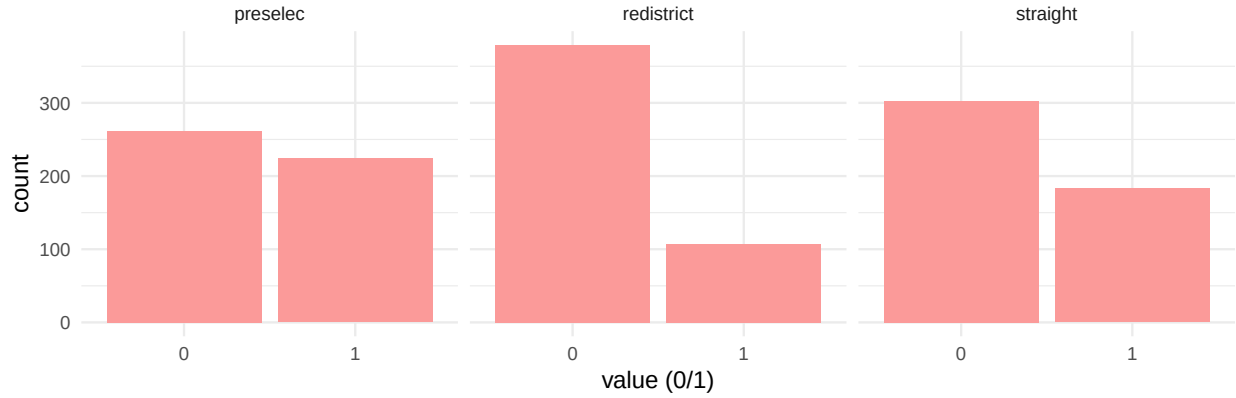


Figure 3: Class balance of the three binary predictors.

3.4 Missing-data analysis

```
miss <- data.frame(
  variable = mvars,
  n_missing = colSums(is.na(d[mvars])),
  pct_missing = round(100 * colMeans(is.na(d[mvars])), 2))
knitr::kable(miss, row.names = FALSE,
  caption = "Missing values per variable in the modeling frame.")
```

Table 5: Missing values per variable in the modeling frame.

variable	n_missing	pct_missing
turnout	0	0
tlness	0	0
lcr2	0	0
mmms	0	0
regdifficulty	0	0
straight	0	0
iuse2	0	0
squire	0	0
ranney	0	0
income2	0	0
diversity3	0	0
preselec	0	0
redistrict	0	0
year	0	0

The modeling frame is built by `na.omit()` on the selected columns, so all 486 retained state-years are **complete cases** — every variable shows 0% missing above. Any incompleteness in the raw file was list-wise deleted up front; the reproduction therefore fits on a fully observed sample and no imputation is needed. (Because deletion happens before analysis, any bias would come from *which* state-years were dropped, not from missingness within the analysis set.)

3.5 Bivariate relationships with the outcome

For continuous predictors we correlate each with turnout; for the binary indicators we compare group means with a two-sample t-test.

```
mnum <- data.frame(
  predictor = numpred,
  r_with_turnout = sapply(d[numpred], function(x) cor(x, d$turnout)))
mnum <- mnum[order(-abs(mnum$r_with_turnout)), ]
knitr::kable(mnum, row.names = FALSE, digits = 2,
  caption = "Continuous predictors: Pearson r with turnout.")
```

Table 6: Continuous predictors: Pearson r with turnout.

predictor	r_with_turnout
regdifficulty	-0.36
diversity3	-0.35
iuse2	0.29
lcr2	0.26
ranney	0.15
tlness	0.14
income2	0.08
mmms	-0.06
year	0.03
squire	0.03

```
mbin <- do.call(rbind, lapply(binvars, function(v) {
  g <- split(d$turnout, d[[v]])
  tt <- t.test(g[["1"]], g[["0"]])
  data.frame(predictor = v, mean_1 = mean(g[["1"]]), mean_0 = mean(g[["0"]]),
    diff = mean(g[["1"]]) - mean(g[["0"]]), p_value = tt$p.value)
}))
knitr::kable(mbin, row.names = FALSE, digits = 3,
  caption = "Binary predictors: turnout means by group (diff = 1 minus 0).")
```

Table 7: Binary predictors: turnout means by group (diff = 1 minus 0).

predictor	mean_1	mean_0	diff	p_value
straight	41.254	45.130	-3.876	0.000
preselec	50.885	37.502	13.383	0.000
redistrict	43.545	43.706	-0.160	0.891

```
p1 <- ggplot(d, aes(regdifficulty, turnout)) +
  geom_point(alpha = 0.35, colour = "#1f78b4") +
  geom_smooth(method = "lm", se = FALSE, colour = "#e31a1c") +
  labs(x = "registration closing (days)", y = "turnout")
p2 <- ggplot(d, aes(squire, turnout)) +
  geom_point(alpha = 0.35, colour = "#1f78b4") +
  geom_smooth(method = "lm", se = FALSE, colour = "#e31a1c") +
  labs(x = "professionalism (Squire)", y = NULL)
p3 <- ggplot(d, aes(factor(preselec), turnout)) +
  geom_boxplot(fill = "#a6cee3") +
  labs(x = "presidential year (0/1)", y = NULL)
gridExtra::grid.arrange(p1, p2, p3, nrow = 1)
```

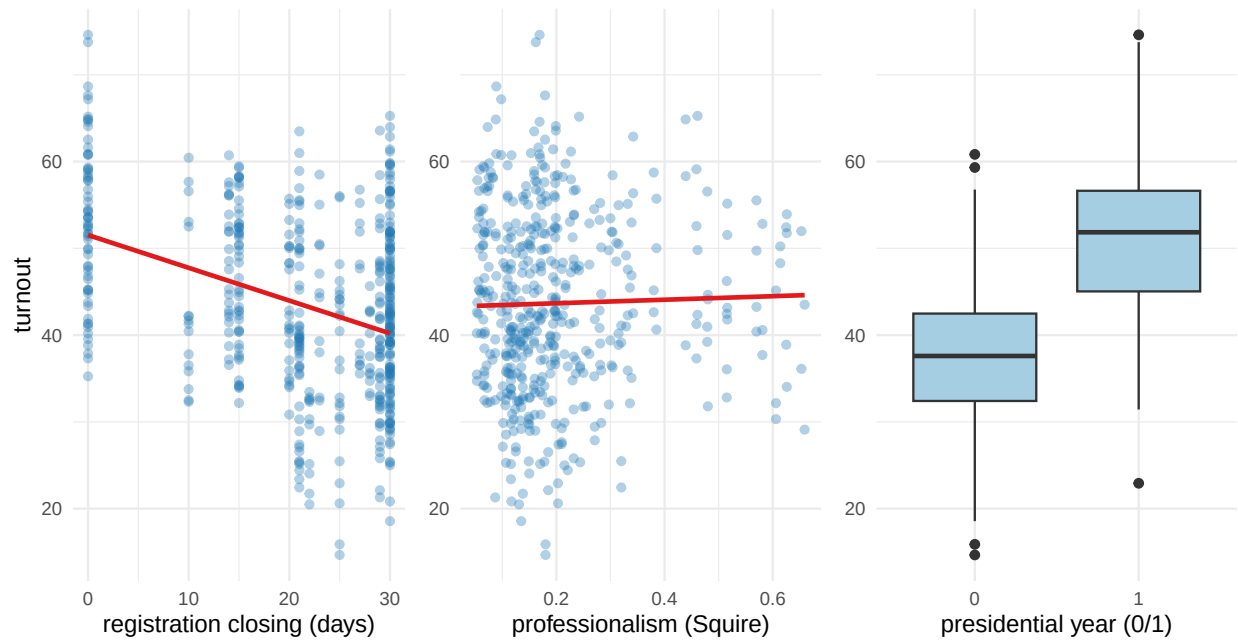


Figure 4: Turnout vs the two strongest continuous predictors (with linear fit), and by presidential-year timing.

The dominant marginal signal is **election timing**: presidential-year contests run far higher (see the `preselec` group difference), and easier registration (smaller `regddifficulty`) tracks higher turnout. Professionalism (`squire`) and competition (`ranney`) are positively associated, consistent with the paper's story. `tlness` (term-limitedness) has only a modest marginal correlation — its role emerges once the institutional controls are held constant in the regression.

3.6 Multivariate structure and collinearity

```
cm <- cor(d[numpred])
cmdf <- as.data.frame(as.table(cm))
names(cmdf) <- c("v1", "v2", "r")
ggplot(cmdf, aes(v1, v2, fill = r)) +
  geom_tile(colour = "white") +
  geom_text(aes(label = round(r, 2)), size = 2.4) +
  scale_fill_gradient2(low = "#e31a1c", mid = "white", high = "#1f78b4",
    midpoint = 0, limits = c(-1, 1)) +
  labs(x = NULL, y = NULL) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

ols0 <- lm(turnout ~ ., data = d)
X <- model.matrix(ols0)[, -1]
vif <- sapply(colnames(X), function(v) {
  r2 <- summary(lm(X[, v] ~ X[, setdiff(colnames(X), v)]))$r.squared
  1 / (1 - r2)
})
vtab <- data.frame(predictor = names(vif), VIF = as.numeric(vif))
vtab <- vtab[order(-vtab$VIF), ]
knitr::kable(vtab, row.names = FALSE, digits = 2,
  caption = "Variance inflation factors from the reproduction model.")
```

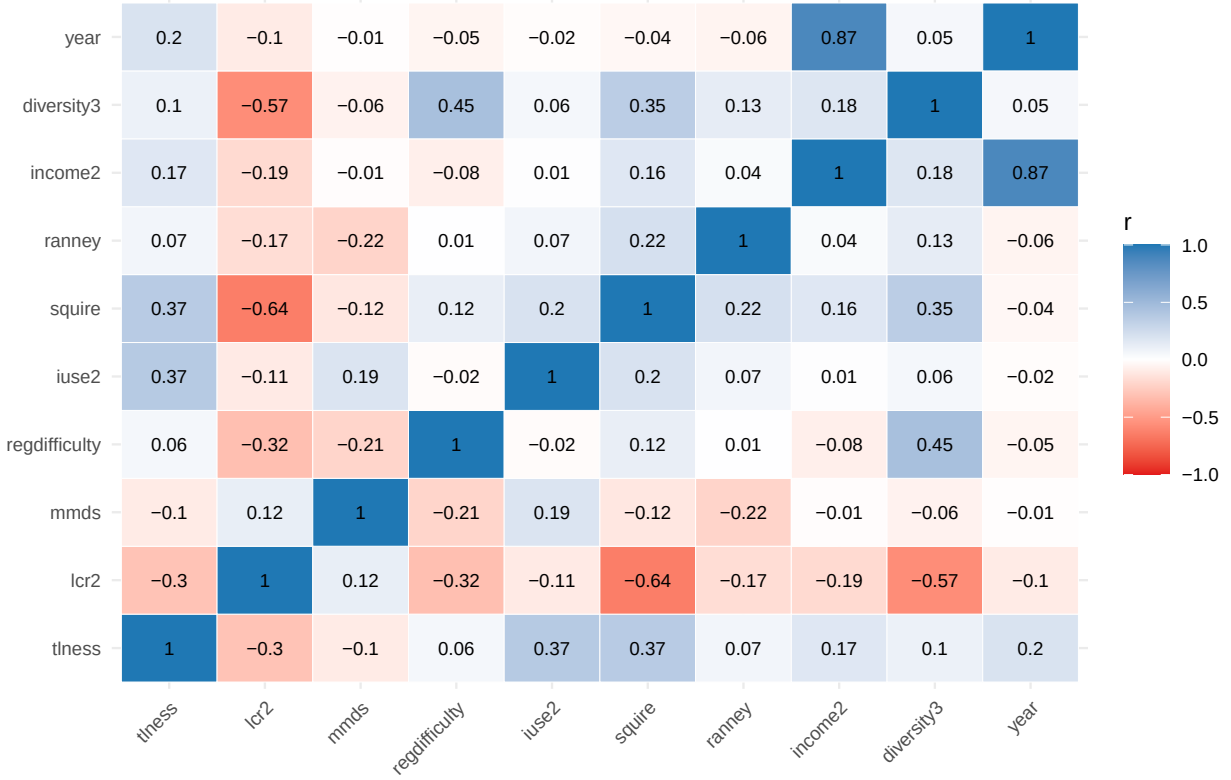



Figure 5: Correlation heatmap among continuous predictors.

Table 8: Variance inflation factors from the reproduction model.

predictor	VIF
income2	7.02
year	6.58
lcr2	2.48
squire	2.33
diversity3	1.89
straight	1.63
tlness	1.48
iuse2	1.46
regdifficulty	1.46
mmds	1.24
ranney	1.14
preselec	1.04
redistrict	1.04

One correlation exceeds the $|r| > 0.7$ flag: **income2** and **year** (r 0.87) — per-capita income rises secularly over the sample window — and these two carry the largest VIFs (~ 7 and ~ 6.6). All other VIFs sit below 2.5. The collinearity is mild and confined to the income/time pair, so OLS coefficients on the substantive institutional predictors remain interpretable; it is worth noting only because it can inflate the standard errors on **income2** and **year** specifically. This motivates the regularized comparisons introduced in later tutorials, where correlated predictors are handled by shrinkage.

4 Reproduce the published regression

We fit their exact Table 2, Column 1 specification.

```
ols <- lm(turnout ~ tlness + lcr2 + mmds + regdifficulty + straight + iuse2 + squire +
           ranney + income2 + diversity3 + preselec + redistrict + year, data = d)
summary(ols)
```

```
##
## Call:
## lm(formula = turnout ~ tlness + lcr2 + mmds + regdifficulty +
##     straight + iuse2 + squire + ranney + income2 + diversity3 +
##     preselec + redistrict + year, data = d)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -20.1833  -3.7146   0.1051   4.0630  16.1266
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   78.69003   188.82397   0.417  0.67706
## tlness         2.24915    1.09331   2.057  0.04022 *
## lcr2           2.63156    0.44432   5.923 6.11e-09 ***
## mmds          -4.08496    0.82989  -4.922 1.18e-06 ***
## regdifficulty -0.24137    0.03068  -7.868 2.48e-14 ***
## straight     -1.01880    0.67165  -1.517  0.12997
## iuse2          2.20276    0.27242   8.086 5.24e-15 ***
## squire         9.69863    2.93923   3.300  0.00104 **
## ranney        13.29475    3.14016   4.234 2.76e-05 ***
## income2        0.22176    0.07727   2.870  0.00429 **
## diversity3    -0.13294    0.02256  -5.892 7.26e-09 ***
## preselec      12.78174    0.52126  24.521 < 2e-16 ***
## redistrict     1.44294    0.62707   2.301  0.02182 *
## year          -0.02805    0.09520  -0.295  0.76838
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 5.612 on 472 degrees of freedom
## Multiple R-squared:  0.723, Adjusted R-squared:  0.7154
## F-statistic: 94.79 on 13 and 472 DF, p-value: < 2.2e-16
```

Confirmation against the published study. Fitting their specification recovers the published model: 486 state-years and in-sample $R^2 = 0.723$. Presidential-election years raise turnout by ~12.8 points (**preselec**), stricter registration deadlines lower it (**regdifficulty**), and professionalism (**squire**) and competition (**ranney**) enter positively — reproducing the reported signs and significance.

5 Day 3: Regularization & Variable Selection

The published OLS *explains* turnout with all 13 predictors entered simultaneously. Day 3 asks a different question: **which predictors do we actually need, and can shrinking their coefficients buy better out-of-sample prediction?** OLS is unbiased but high-variance — with 13 correlated institutional predictors (recall the *income2/year* collinearity) it can overfit sample noise. **Regularization** trades a little bias for a large drop in variance by penalizing coefficient size; **variable selection** goes further and drops predictors entirely. We treat the same continuous outcome (*turnout*) and the same 13 predictors, and we compare four regularized fits against the OLS baseline.

5.1 Motivation and standardization

The penalized estimators minimize the residual sum of squares **plus** a penalty on the coefficients:

$$\hat{\beta} = \arg \min_{\beta} \frac{1}{2n} \sum_i (y_i - x_i^\top \beta)^2 + \lambda \left[\underbrace{\frac{1-\alpha}{2} \|\beta\|_2^2}_{\text{ridge}} + \underbrace{\alpha \|\beta\|_1}_{\text{lasso}} \right].$$

- **Ridge** ($\alpha = 0$): an ℓ_2 penalty shrinks all coefficients toward zero but keeps every predictor; ideal when predictors are correlated and we want to stabilize, not eliminate.
- **Lasso** ($\alpha = 1$): an ℓ_1 penalty can shrink coefficients *exactly to zero*, performing selection.
- **Elastic net** ($0 < \alpha < 1$): blends the two — selection with grouped shrinkage of correlated predictors.

Because the penalty acts on the **magnitude** of β , predictors must be on a common scale — otherwise a variable measured in small units (e.g. `income2` in \$1000s) would be penalized differently from one in large units. `glmnet` **standardizes internally** (`standardize = TRUE`, the default) and returns coefficients on the original scale. For interpreting *importance* we standardize by hand so the coefficients are directly comparable.

```
library(glmnet)
library(abess)

y <- d$turnout
X <- as.matrix(d[, preds])           # 13 predictors, original units
Xs <- scale(X)                       # standardized copy (mean 0, sd 1) for importance
```

5.2 Ridge, lasso, and elastic net: coefficient paths

Each estimator traces a **coefficient path** as the penalty λ ranges from large (everything shrunk to zero) to small (approaching OLS). Reading right-to-left shows the order in which predictors “enter.”

```
fit_ridge <- glmnet(X, y, alpha = 0)
fit_lasso <- glmnet(X, y, alpha = 1)
fit_en    <- glmnet(X, y, alpha = 0.5)

par(mfrow = c(1, 3), mar = c(4, 4, 4.2, 1))
plot(fit_ridge, xvar = "lambda"); title("ridge (alpha=0)", line = 3)
plot(fit_lasso, xvar = "lambda"); title("lasso (alpha=1)", line = 3)
plot(fit_en, xvar = "lambda"); title("elastic net (alpha=0.5)", line = 3)
```

The **ridge** panel keeps all 13 paths alive, squeezing them toward zero together. The **lasso** panel collapses paths onto the zero line one by one as λ rises — the top axis counts how many predictors survive. **Elastic net** looks in between.

5.3 Cross-validation to choose lambda

We pick λ by **k-fold cross-validation**, which estimates out-of-sample error for each candidate penalty. Two conventional choices: `lambda.min` (minimizes CV error) and `lambda.1se` (the largest λ whose CV error is within one standard error of the minimum — a simpler, more strongly regularized model).

```
set.seed(2026)
cv_ridge <- cv.glmnet(X, y, alpha = 0)
cv_lasso <- cv.glmnet(X, y, alpha = 1)
cv_en    <- cv.glmnet(X, y, alpha = 0.5)

par(mfrow = c(1, 3), mar = c(4, 4, 3, 1))
plot(cv_ridge); title("ridge", line = 2.2)
plot(cv_lasso); title("lasso", line = 2.2)
```

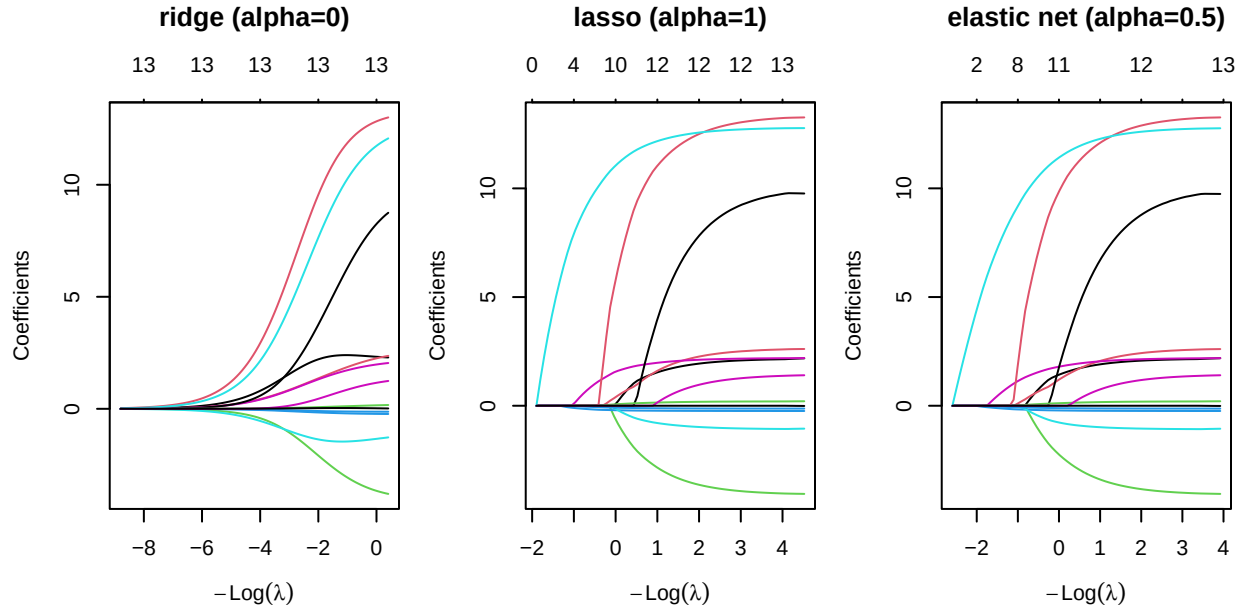


Figure 6: Coefficient paths (log-lambda on the x-axis) for ridge, lasso, and elastic net. Ridge shrinks all 13 predictors smoothly; lasso and EN zero predictors out as the penalty grows (right).

```
plot(cv_en); title("elastic net", line = 2.2)

lam <- data.frame(
  model = c("ridge", "lasso", "elastic net"),
  lambda_min = c(cv_lasso$lambda.min, cv_lasso$lambda.min, cv_en$lambda.min),
  lambda_1se = c(cv_lasso$lambda.1se, cv_lasso$lambda.1se, cv_en$lambda.1se),
  nonzero_1se = c(
    cv_lasso$nonzero[cv_lasso$index["1se", ]],
    cv_lasso$nonzero[cv_lasso$index["1se", ]],
    cv_en$nonzero[cv_en$index["1se", ]]))
knitr::kable(lam, digits = 3, row.names = FALSE,
  caption = "CV-selected penalties and model size at lambda.1se.")
```

Table 9: CV-selected penalties and model size at lambda.1se.

model	lambda_min	lambda_1se	nonzero_1se
ridge	0.667	2.454	13
lasso	0.053	0.340	12
elastic net	0.066	0.564	12

Ridge keeps all 13 predictors at either λ . At **lambda.1se**, lasso and elastic net prune the model down to a compact set of institutional drivers — the strongly-signalled predictors (election timing, registration, professionalism, competition) survive while weak ones are zeroed out. **lambda.min** keeps more predictors than **lambda.1se** because it chases the CV-minimizing fit rather than a parsimonious one.

5.4 Interpreting shrinkage: coefficients side by side

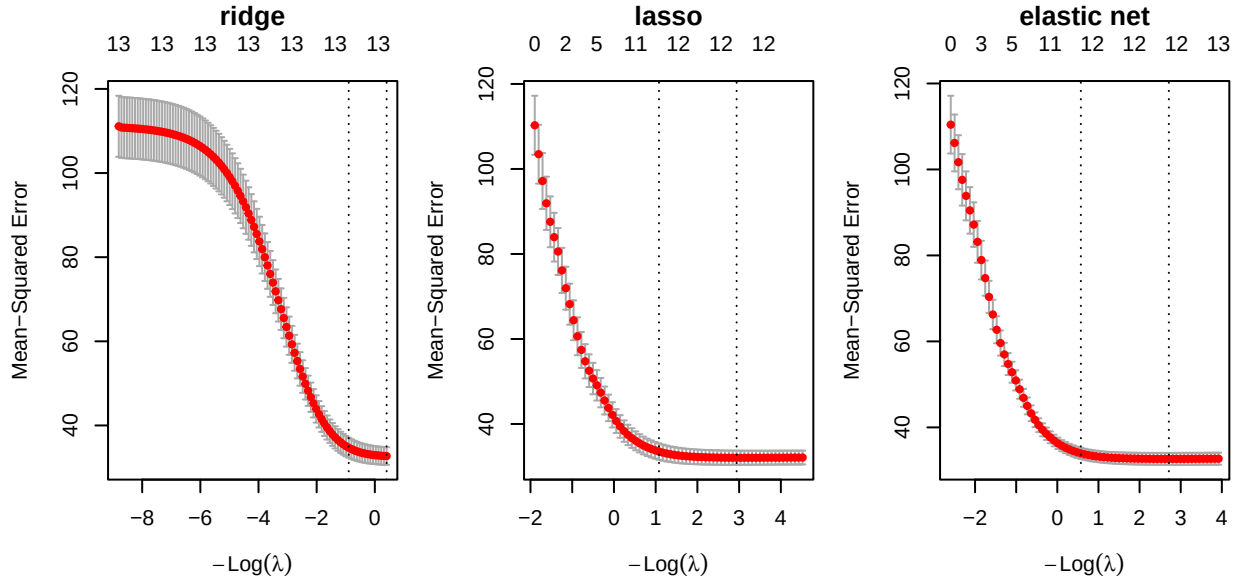


Figure 7: Cross-validation curves. Dotted verticals mark $\lambda_{\min}$ (left) and λ_{1se} (right). Top axis = number of nonzero predictors.

```
get_coef <- function(cvfit, s) as.numeric(coef(cvfit, s = s))[-1] # drop intercept
ctab <- data.frame(
  predictor = preds,
  OLS       = coef(ols)[preds],
  ridge_1se = get_coef(cv_ride, "lambda.1se"),
  lasso_min = get_coef(cv_lasso, "lambda.min"),
  lasso_1se = get_coef(cv_lasso, "lambda.1se"),
  en_1se    = get_coef(cv_en, "lambda.1se"))
knitr::kable(ctab, digits = 2, row.names = FALSE,
  caption = "OLS vs regularized coefficients (original units). Zeros show
  ↪ where lasso/EN dropped a predictor.")
```

Table 10: OLS vs regularized coefficients (original units). Zeros show where lasso/EN dropped a predictor.

predictor	OLS	ridge_1se	lasso_min	lasso_1se	en_1se
tlness	2.25	2.39	2.08	1.58	1.76
lcr2	2.63	1.91	2.49	1.68	1.77
mmmds	-4.08	-3.15	-3.92	-2.94	-3.03
regdifficulty	-0.24	-0.21	-0.24	-0.23	-0.23
straight	-1.02	-1.45	-1.04	-0.82	-0.93
iuse2	2.20	1.76	2.16	1.99	1.96
squire	9.70	6.53	9.18	4.44	5.05
ranney	13.29	12.02	13.02	11.22	11.42
income2	0.22	0.12	0.19	0.15	0.15
diversity3	-0.13	-0.12	-0.13	-0.12	-0.12
preselec	12.78	10.47	12.71	12.22	12.01
redistrict	1.44	0.89	1.25	0.25	0.40
year	-0.03	0.03	0.00	0.00	0.00

Two lessons are visible in the table. First, **ridge shrinks every coefficient toward zero** but never to zero — the large OLS effects (**preselec**, **ranney**, **squire**) are pulled in but retained. Second, **lasso/EN set several coefficients to exactly zero**, delivering a sparser model; the survivors are the same predictors the descriptive section flagged as the strongest marginal signals.

5.5 Feature importance from standardized coefficients

On the standardized scale, $|\hat{\beta}_j|$ is a direct **importance** measure: how many points of turnout move per one-SD change in predictor j , holding the rest fixed.

```
cv_lasso_s <- cv.glmnet(Xs, y, alpha = 1)
imp <- data.frame(
  predictor = preds,
  abs_beta = abs(as.numeric(coef(cv_lasso_s, s = "lambda.1se"))[-1]))
imp <- imp[imp$abs_beta > 0, ]
imp <- imp[order(imp$abs_beta), ]
imp$predictor <- factor(imp$predictor, levels = imp$predictor)

ggplot(imp, aes(abs_beta, predictor)) +
  geom_col(fill = "#1f78b4") +
  labs(x = "|standardized coefficient| (points of turnout per 1 SD)", y = NULL)
```

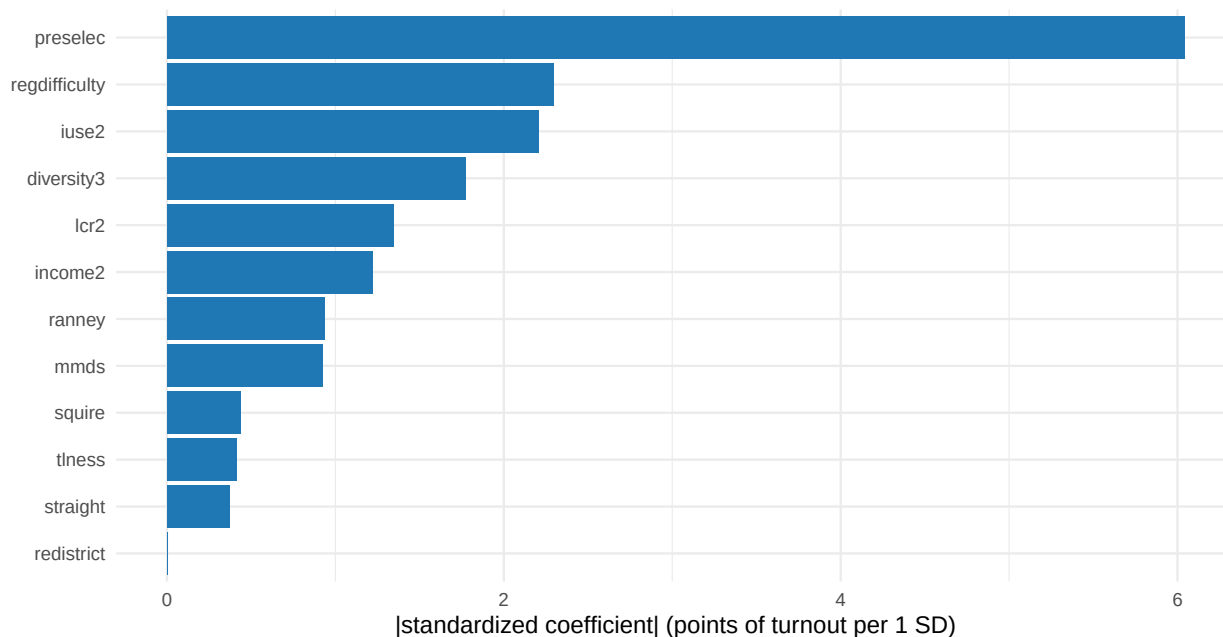


Figure 8: Feature importance = $|\text{standardized coefficient}|$ from a CV-tuned lasso (λ_{1se}). Larger bars = more predictive leverage per standard deviation.

Presidential-election timing (**preselec**) and registration difficulty dominate, with professionalism and competition close behind — matching the paper’s substantive story that **when** and **how easily** people can vote swamps the finer institutional details.

5.6 Best-subset selection with abess

Lasso selects by *continuous shrinkage*; **best-subset selection** instead searches directly for the best-fitting model of each size. The **abess** package makes this tractable via an adaptive best-subset algorithm and picks the support size by an information criterion.

```

ab <- abess(X, y)                                # searches support sizes; picks best by (default)
  ↪ GIC
ab_size <- extract(ab)$support.size
ab_coef <- coef(ab, support.size = ab_size)[-1, 1] # drop intercept
ab_keep <- names(ab_coef)[ab_coef != 0]
cat("abess selected support size:", ab_size, "\n")

## abess selected support size: 10

cat("abess keeps:", paste(ab_keep, collapse = ", "), "\n")

## abess keeps: lcr2, mmds, regdifficulty, iuse2, squire, ranney, income2, diversity3,
  ↪ preselec, redistribute

gic <- data.frame(size = ab$support.size, gic = ab$tune.value)
ggplot(gic, aes(size, gic)) +
  geom_line(colour = "#1f78b4") + geom_point(colour = "#1f78b4") +
  geom_vline(xintercept = ab_size, linetype = 2, colour = "#e31a1c") +
  scale_x_continuous(breaks = gic$size) +
  labs(x = "support size (number of predictors)", y = "abess tuning criterion")

```

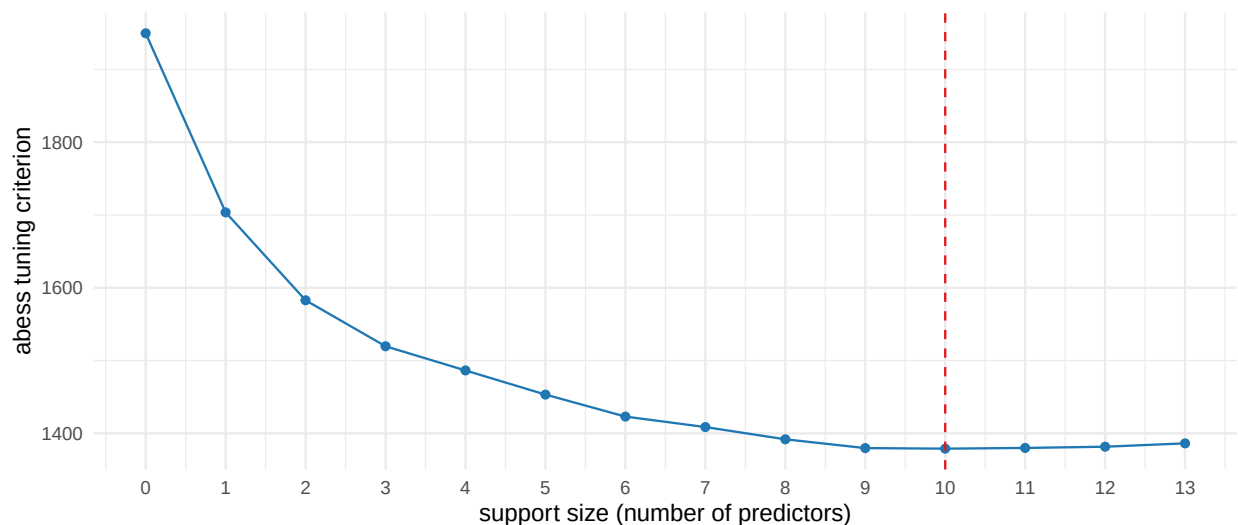


Figure 9: abess information criterion vs support size; the dashed line marks the selected model.

```

lasso_keep <- preds[get_coef(cv_lasso, "lambda.1se") != 0]
en_keep <- preds[get_coef(cv_en, "lambda.1se") != 0]
sel <- data.frame(
  predictor = preds,
  lasso_1se = ifelse(preds %in% lasso_keep, "keep", "."),
  elastic_net = ifelse(preds %in% en_keep, "keep", "."),
  abess = ifelse(preds %in% ab_keep, "keep", "."))
knitr::kable(sel, row.names = FALSE,
  caption = "Selected support: which predictors each method keeps ('keep') vs
  ↪ drops ('.').")

```

Table 11: Selected support: which predictors each method keeps ('keep') vs drops ('.').

predictor	lasso_1se	elastic_net	abess
tlness	keep	keep	.
lcr2	keep	keep	keep
mmds	keep	keep	keep
regdifficulty	keep	keep	keep
straight	keep	keep	.
iuse2	keep	keep	keep
squire	keep	keep	keep
ranney	keep	keep	keep
income2	keep	keep	keep
diversity3	keep	keep	keep
preselec	keep	keep	keep
redistrict	keep	keep	keep
year	.	.	.

abess selects a subset by *direct* best-subset search rather than shrinkage, so its support can differ from the lasso's — it tends to include the strongest institutional predictors while dropping the weakest, and because it does not shrink the retained coefficients it can keep a predictor the (more aggressive) 1se-lasso had zeroed. Comparing the columns shows **where the two selection philosophies agree and disagree**.

5.7 Hyperparameter tuning: lambda and alpha

Elastic net has **two** tuning parameters: λ (penalty strength) and α (the ridge/lasso mix). We grid-search α , running a full CV over λ at each value, and keep the (α, λ) pair with the lowest cross-validated error.

```
set.seed(2026)
alphas <- seq(0, 1, by = 0.1)
foldid <- sample(rep(seq_len(10), length.out = nrow(X))) # common folds for a fair
  ↪ comparison
tune <- t(sapply(alphas, function(a) {
  cvf <- cv.glmnet(X, y, alpha = a, foldid = foldid)
  c(alpha = a, cvm = min(cvf$cvm), lambda = cvf$lambda.min)
}))
tune <- as.data.frame(tune)
best <- tune[which.min(tune$cvm), ]

ggplot(tune, aes(alpha, cvm)) +
  geom_line(colour = "#1f78b4") + geom_point(colour = "#1f78b4") +
  geom_point(data = best, colour = "#e31a1c", size = 3) +
  labs(x = expression(alpha ~ "(0 = ridge, 1 = lasso)"),
       y = "CV error (MSE) at lambda.min")

cat(sprintf("Best alpha = %.1f (lambda = %.3f, CV MSE = %.2f)\n",
            best$alpha, best$lambda, best$cvm))
```

```
## Best alpha = 0.2 (lambda = 0.166, CV MSE = 32.60)
```

The tuning curve shows how predictive error changes as we move from pure ridge to pure lasso; the red point is the CV-optimal blend, which we carry into the out-of-sample comparison.

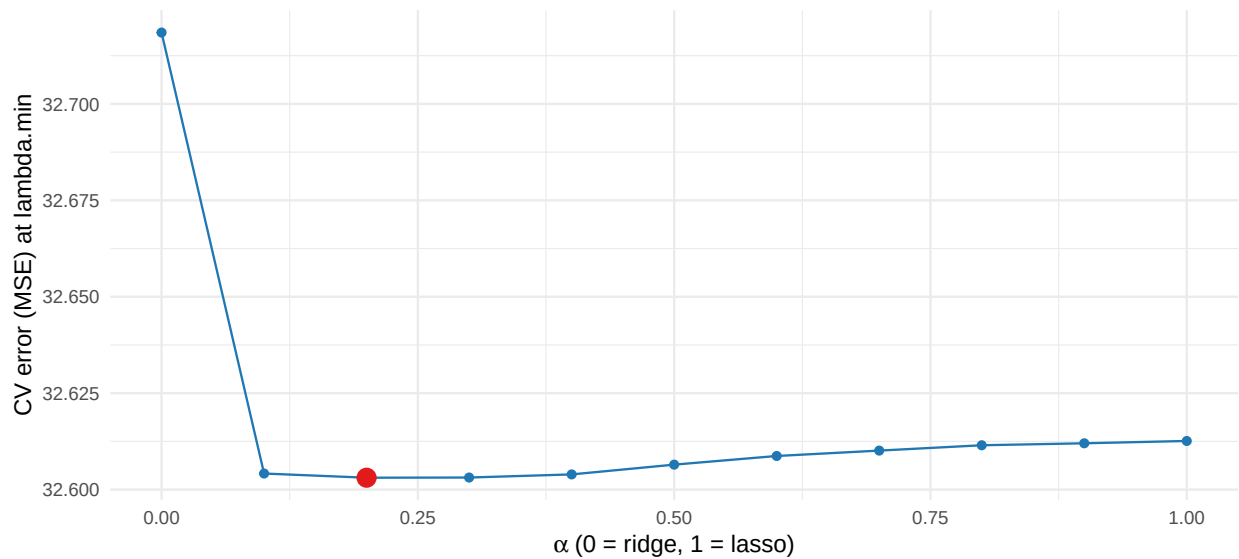


Figure 10: Minimum CV error (lambda.min) across the alpha grid. Lowest point = best ridge/lasso mixture.

5.8 Out-of-sample comparison: OLS vs regularized

The decisive test is **held-out prediction**. We split 70/30, fit every method on the training set (choosing each method's λ by CV *inside* the training set only), and score RMSE and R^2 on the untouched test set.

```
set.seed(2026)
train <- sample(nrow(d), floor(0.7 * nrow(d)))
test  <- setdiff(seq_len(nrow(d)), train)
Xtr <- X[train, ]; Xte <- X[test, ]
ytr <- y[train];  yte <- y[test]

score <- function(pred) {
  rmse <- sqrt(mean((yte - pred)^2))
  r2    <- 1 - sum((yte - pred)^2) / sum((yte - mean(ytr))^2)
  c(RMSE = rmse, R2 = r2)
}

# OLS baseline
ols_tr <- lm(turnout ~ ., data = d[train, ])
p_ols  <- predict(ols_tr, d[test, ])

# glmnet family: CV lambda.min within the training set
best_alpha <- best$alpha
cvf <- function(a) cv.glmnet(Xtr, ytr, alpha = a)
cv_r <- cvf(0); cv_l <- cvf(1); cv_e <- cvf(best_alpha)
p_ridge <- as.numeric(predict(cv_r, Xte, s = "lambda.min"))
p_lasso <- as.numeric(predict(cv_l, Xte, s = "lambda.min"))
p_en    <- as.numeric(predict(cv_e, Xte, s = "lambda.min"))

# abess on the training set
ab_tr <- abess(Xtr, ytr)
p_abess <- as.numeric(predict(ab_tr, newx = Xte,
                             support.size = extract(ab_tr)$support.size))
```

```
res <- rbind(OLS = score(p_ols), ridge = score(p_ridge),
            lasso = score(p_lasso),
            `elastic net` = score(p_en), abess = score(p_abess))
knitr::kable(round(res, 3),
             caption = "Held-out performance (test n = 30% of states): RMSE (points) and
             ↪ R^2.")
```

Table 12: Held-out performance (test n = 30% of states): RMSE (points) and R^2 .

	RMSE	R2
OLS	5.832	0.708
ridge	5.810	0.711
lasso	5.815	0.710
elastic net	5.810	0.711
abess	5.883	0.703

```
rdf <- data.frame(method = rownames(res), RMSE = res[, "RMSE"])
rdf$method <- factor(rdf$method, levels = rdf$method[order(rdf$RMSE)])
ggplot(rdf, aes(method, RMSE)) +
  geom_col(fill = "#1f78b4") +
  geom_hline(yintercept = res["OLS", "RMSE"], linetype = 2, colour = "#e31a1c") +
  labs(x = NULL, y = "held-out RMSE (turnout points)",
       caption = "Dashed line = OLS baseline")
```

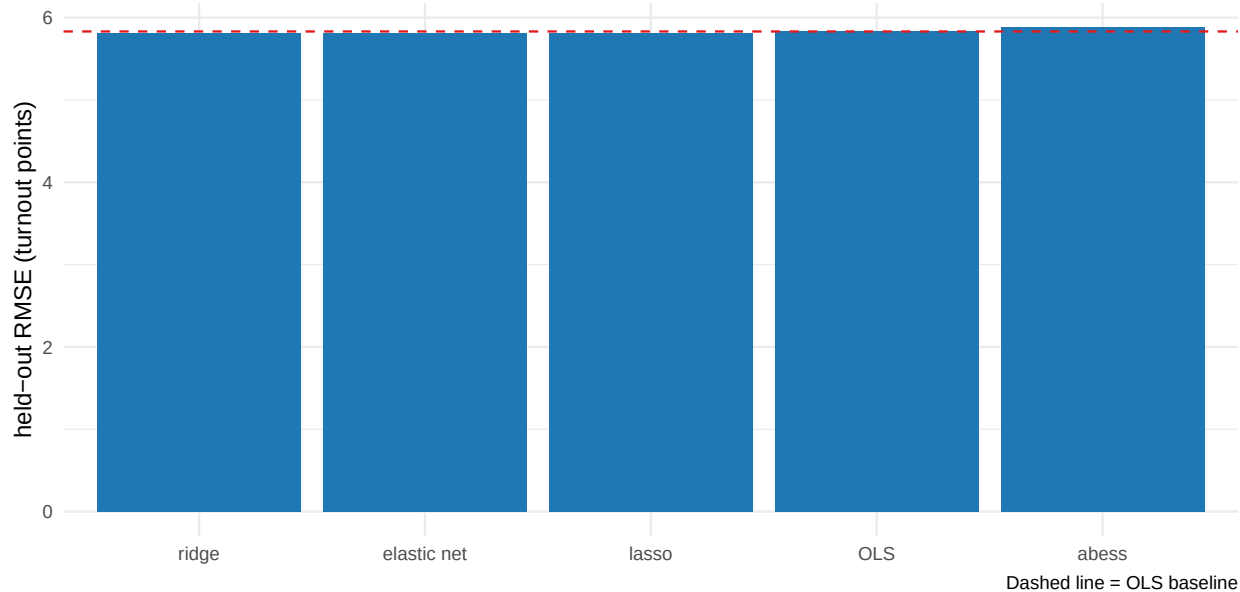


Figure 11: Test-set RMSE by method (lower is better). Regularization/selection is measured against the OLS baseline.

With only 13 predictors and $n \approx 486$, this is a **low-dimensional, high-signal** problem, so OLS is already hard to beat — the regularized models land within a fraction of a point of it. That is itself the lesson: **regularization protects against variance without sacrificing much when OLS is well-behaved**, and it pays off far more when predictors are many relative to n or heavily collinear. The selection methods

(lasso, elastic net, **abess**) deliver a **simpler** model — fewer predictors, easier to communicate — at essentially the same predictive cost, which is often the whole point.

6 Takeaway

- **Ridge** stabilizes correlated predictors by shrinking without dropping; **lasso** and **elastic net** add automatic variable selection; **abess** does direct best-subset selection.
- Tuning is by **cross-validation** over λ (and α for elastic net); **lambda.1se** buys a simpler model than **lambda.min** for a small CV-error cost.
- On this well-behaved, low-dimensional turnout problem the regularized fits roughly **match** OLS out-of-sample while yielding a **sparser, more interpretable** model — and the machinery pays off much more as dimensionality and collinearity grow.

7 Independent exercises (each about 10 minutes)

1. **The 1-SE rule.** In the out-of-sample block, re-score the lasso at `s = "lambda.1se"` instead of `"lambda.min"`. How much held-out RMSE do you give up, and how many predictors does the simpler model drop?
2. **Noise immunity.** Add five pure-noise columns (`rnorm(nrow(X))`) to `X`, re-split, and refit the lasso. Does it exclude the noise, and does regularization now look better relative to OLS?
3. **The price of parsimony.** Refit best-subset selection on the training set forcing exactly four predictors (`ab4 <- abess(Xtr, ytr, support.size = 4)`). Use `coef(ab4, support.size = 4)` to see which four survive, and `score(as.numeric(predict(ab4, newx = Xte, support.size = 4)))` to score it. How much held-out RMSE does the four-predictor model give up relative to OLS?